Databases 1

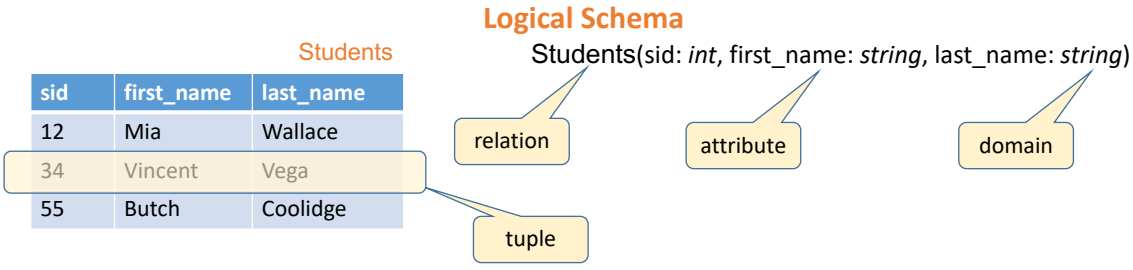
dr hab. inż. Przemysław Korytkowski, prof. ZUT

Lecture 3 – Relational model and introduction to SQL

1

Relational model

- A database is a collection of one or more **relations**, where each relation is a **table** with **rows** and **columns**.
- In the formal relational model terminology, a row is called a **tuple**, a column header is called an **attribute**, and the table is called a **relation**. The data type describing the types of values that can appear in each column is represented by a **domain** of possible values.



2

Relational model

- A **relation schema**: $R_k(A_1, A_2, \ldots, A_n)$
- **Domain of attribute** $R_k.A_i: \mathrm{dom}(A_i)$
  $R_k(A_1: \mathrm{dom}(A_1), A_2: \mathrm{dom}(A_2), \ldots, A_n: \mathrm{dom}(A_n))$
- A **domain** is a set of atomic values (each value in the domain is indivisible). A domain is a specified **data type** (integer, float, string, character, bit) from which the data values forming the domain are drawn.
- The **degree of a relation** is the number of attributes.
- **Relation** or **relations state** $r(R_k)$ of the relations schema $R_k(A_1, A_2, \ldots, A_n)$ is a **set of n-tuples** $r = r(R_k) = \{t_1, t_2, \ldots, t_m\}$
- Each n-tuple t is an ordered list of n values $t = \langle v_1, v_2, \ldots, v_n \rangle$, where each value $v_i \in \mathrm{dom}(A_i)$ or is a special **NULL** value.

3

Relational model

- A relation is defined as a set of tuples $r = \{t_1, t_2, \ldots, t_m\}$. Thus, the definition of a relations does not specify any order.
- Mathematically, elements of a set have no order among them. Hence, tuples in a relations do not have any order.
- N-tuples form a set; hence they cannot be repeated.

- n-tuple is an ordered list of n values $t = \langle v_1, v_2, \ldots, v_n \rangle$, hence the ordering of attributes in a relations is important.
- $t_1 = \langle 12, 'Mia', 'Wallace' \rangle, t_2 = \langle 34, 'Vincent', 'Vega' \rangle, t_3 = \langle 55, 'Butch', 'Coolidge' \rangle$
- $r = \{\langle 12, 'Mia', 'Wallace' \rangle, \langle 55, 'Butch', 'Coolidge' \rangle, \langle 34, 'Vincent', 'Vega' \rangle\}$
- $R(\mathrm{SID}, \mathrm{first\_name}, \mathrm{last\_name})$

4

Relational model – superkey

- A relation is defined as a set of tuples. All elements of a set are distinct; hence, all tuples in a relation must also be distinct.

Super key

- $t_i, t_j \in r(R), i \neq j$
- $SK = \{B_1, B_2, \ldots, B_k\} \subseteq \{A_1, A_2, \ldots, A_n\}$
- $\vee\, t_i[SK] \neq t_j[SK]$

Subset of attributes

uniqueness constraint – no two distinct tuples in any state r of R can have the same value for SK.

5

Relational model – superkey

Person(ID, passportNumber, FirstName, LastName)

Superkeys:

- {ID, passportNumber, FirstName, LastName}
- {ID, passportNumber, LastName}
- {ID, LastName}
- {passportNumber, LastName}
- {passportNumber}
- {ID}

6

Relational model – key

Key:

- is a minimal superkey — a superkey from which we cannot remove any attributes and still have the uniqueness constraint hold.
- is used to identify tuples in the relation.

- A relation schema may have more than one key.
- Each of the keys is called a **candidate key**.
- One of the candidate keys as the **primary key** of the relation.
- The choice of one to become the primary key is somewhat arbitrary.

  - {passportNumber}
  - {ID}

The best because it's artificial values.

7

Relational model – database schema

- A **relational database schema** S is a set of relation schemas $S = \{R_1, R_2, \ldots, R_m\}$ and a set of integrity constraints IC.

- A **relational database state** DB of S is a set of relation states $DB = \{r_1, r_2, \ldots, r_m\}$ such that each r_i is a state of R_i and such that the r_i relation states satisfy the integrity constraints specified in IC.

8

Relational model – integrity constraints

The **entity integrity constraint** states that no primary key value can be NULL.

The **referential integrity constraint** is specified between two relations and is used to maintain the consistency among tuples in the two relations.

A set of attributes FK in relation schema R_1(referencing relation) is a **foreign key** of R_2 (referenced relation) that references relation R_2 if it satisfies the following rules:

1. The attributes in FK have the same domain(s) as the primary key attributes PK of R_2.
2. $t_1[FK] = t_2[PK]$ or $t_1[FK]$ is Null.

9

Referential integrity constraint

- Referential integrity constraint is depicted as directed arc from each foreign key to the relation it references.

Students

sid	first_name	last_name
12	Mia	Wallace
34	Vincent	Vega
55	Butch	Coolidge

Enrolled

sid	cid	grade
12	CS101	NULL
12	CS900	5.0
55	CS900	NULL
34	CS102	3.5

Courses

cid	cname	ECTS
CS101	Programming	5
CS900	Databases	6
CS102	Algorithms	4
EN101	Mechanics	5

10

Foreign key violations

- Insert a tuple (record) with foreign key value that does not appear in referenced relation (table)
  - DBMS rejects insert
- What should happen to referencing relation tuples when in referenced relation a value is deleted or updated:
  - Delete/update all tuples with the value in referencing relation.
  - Disallow deletion/update
  - Set to Null all tuples with the value in referencing relation.
  - Set to default all tuples with the value in referencing relation.

Deleted record

SID	first_name	last_name
12	Mia	Wallace
34	Vincent	Vega
55	Butch	Coolidge

SID	cid	grade
12	CS101	NULL
12	CS900	5.0
55	CS900	NULL
34	CS102	3.5

?

11

Constraints violation

Insert - operation provides a list of attribute values for a new tuple *t* that is to be inserted into a relation *R*.

- **Domain constraints** can be violated if an attribute value is given that does not appear in the corresponding domain or is not of the appropriate data type.
- **Entity integrity** can be violated if any part of the primary key of the new tuple *t* is NULL.
- **Key constraints** can be violated if a key value in the new tuple *t* already exists in another tuple in the relation *r(R)*.
- **Referential integrity** can be violated if the value of any foreign key in *t* refers to a tuple that does not exist in the referenced relation.

If one or more constraints is violated, the DBMS rejects the insertion.

12

Constraints violation

The **Delete** operation can violate only referential integrity – the tuple being deleted is referenced by foreign keys from other tuples in the database.

Updating:

- Updating an attribute that is neither part of a primary key nor part of a foreign key usually causes no problems; the DBMS need only check to confirm that the new value is of the correct data type and domain.
- Modifying a primary key value is like deleting one tuple and inserting another in its place because we use the primary key to identify tuples.
- If a foreign key attribute is modified, the DBMS must make sure that the new value refers to an existing tuple in the referenced relation (or is set to NULL).

13

Relational model - transactions

- A transaction is an executing operations (reading, inserts, deletions, updates) on the database.
- At the end of the transaction, it must leave the database in a valid or consistent state that satisfies all the constraints specified on the database schema.
- A single transaction may involve any number of operations. These operations together form an **atomic unit of work** against the database.

Relational database running transactions is called **online transaction processing (OLTP)**

14

SQL – Structured Query Language

- SQL is a declarative programming language (4th generation language) used by nearly all relational databases to query, manipulate, and define data, and to provide access control.
- SQL was first developed at IBM by Donald D. Chamberlin and Raymond F. Boyce in the 1970s based on Codd's relational model, which led to implementation of the SQL ANSI standard.
- Versions/Standards:
  - SQL-86
  - SQL-89
  - SQL-92, aka SQL2
  - SQL-99, aka SQL3
  - SQL-03 and SQL-06 – added XML

Each database vendor has its own version of SQL that is at least SQL-99 compliant in the vast majority of queries, but the code in SQL is not directly portable.

15

15

SQL – Structured Query Language

- The **Data Definition Language** (DDL): This subset of SQL supports the creation, deletion, and modification of definitions for tables and views. Integrity constraints can be defined on tables, either when the table is created or later.
- The **Data Manipulation Language** (DML): This subset of SQL allows users to insert, delete, and modify tuples.
- **Triggers** and **Advanced Integrity Constraints**: Triggers are actions executed by the DBMS whenever changes to the database meet conditions specified in the trigger. SQL allows the use of queries to specify complex integrity constraint specifications.
- **Transaction Management**: Various commands allow a user to explicitly control aspects of how a transaction is to be executed.
- **Security**: SQL provides mechanisms to control users' access to data objects such as tables and views.

16

SQL vs. relational model

- In relational model table is a **set of tuples**.
- In SQL table is a **multiset** (sometimes called a bag) **of tuples**.
- Some SQL relations are constrained to be sets because a key constraint has been declared or because the DISTINCT option has been used with the SELECT statement

Relational model

set

CID	cname	ECTS
CS101	Programming	5
CS900	Databases	6
CS102	Algorithms	4
EN101	Mechanics	5

SQL

multiset

CID	cname	ECTS
CS101	Programming	5
CS900	Databases	6
CS102	Algorithms	4
EN101	Mechanics	5
CS900	Databases	6

17

SQL – data types

Atomic data types available for attributes:

- numeric: INT, FLOAT, DOUBLE, DECIMAL(i, j), BIT
- string: CHAR(n), VARCHAR(n)
- date and time: DATE, DATETIME, TIMESTAMP
- semi-structured: JSON, XML
- spatial: POINT, POLYGON

Type	Storage (Bytes)	Minimum Value Signed	Minimum Value Unsigned	Maximum Value Signed	Maximum Value Unsigned
TINYINT	1	-128	0	127	255
SMALLINT	2	-32768	0	32767	65535
MEDIUMINT	3	-8388608	0	8388607	16777215
INT	4	-2147483648	0	2147483647	4294967295
BIGINT	8	-2^{63}	0	$2^{63}-1$	$2^{64}-1$

Value	CHAR(4)	Storage Required	VARCHAR(4)	Storage Required
''	' '	4 bytes	''	1 byte
'ab'	'ab '	4 bytes	'ab'	3 bytes
'abcd'	'abcd'	4 bytes	'abcd'	5 bytes
'abcdefgh'	'abcd'	4 bytes	'abcd'	5 bytes

18

SQL – constraints

```
CREATE TABLE Students (
  SID          INT PRIMARY KEY CHECK (SID > 0),
  first_name   VARCHAR(20) DEFAULT 'Eva',
  last_name    VARCHAR(30) NOT NULL,
  student_card_d INT UNIQUE
  CONSTRAINT positiveCardID CHECK (student_card_d > 0)
);
```

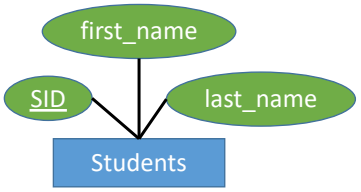
Named constraint

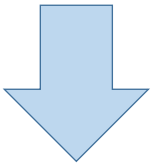
- In practice, we don't specify many constraints.
- Constraints deteriorate performance.

19

From ERD to SQL

- An entity set becomes a relation (table)





```
CREATE TABLE Students (
  SID          INT PRIMARY KEY,
  first_name   VARCHAR(20),
  last_name    VARCHAR(30)
);
```

Students

SID	first_name	last_name

20

20

From ERD to SQL

1:N

```
CREATE TABLE Professors (
 PID         INT PRIMARY KEY,
 first_name VARCHAR(20),
 last_name  VARCHAR(30)
);
```

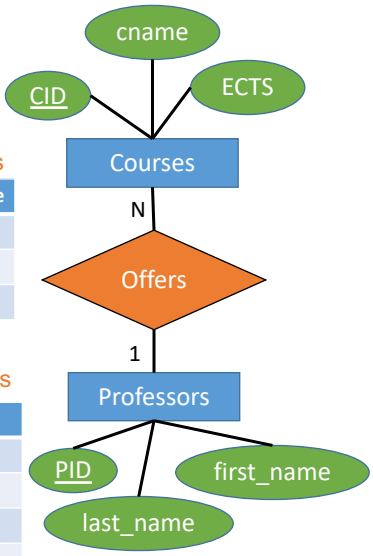
```
CREATE TABLE Courses(
 CID         INT PRIMAY KEY
 cname       CHAR(50),
 ECTS        INT,
 PID         INT,
 FOREIGN KEY (PID)
   REFERENCES Professors(PID)
);
```

Professors

PID	first_name	last_name

Courses

CID	cname	ECTS	PID

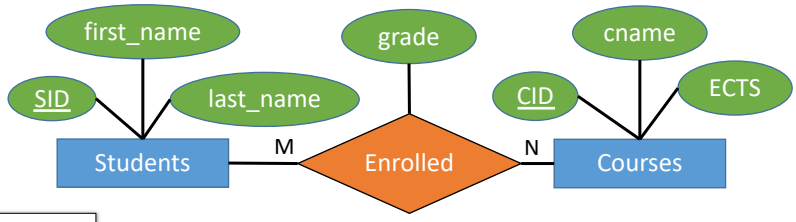


21

21

From ERD to SQL

M:N




```
CREATE TABLE Enrolled(
  SID    INT,
  CID    CHAR(10),
  grade  FLOAT,
  PRIMARY KEY (SID, CID),
  FOREIGN KEY (SID)
      REFERENCES Students(SID),
  FOREIGN KEY (CID)
      REFERENCES Courses(CID)
);
```

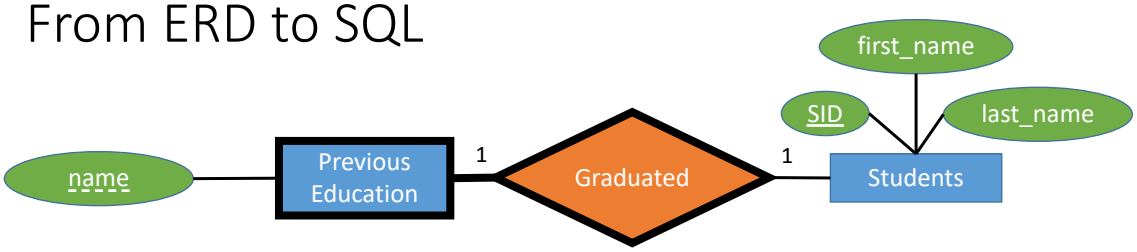
Enrolled

SID	CID	grade

22

22

From ERD to SQL




```
CREATE TABLE PreviousEducation(
  SID       INT NOT NULL,
  name      CHAR(50),
  FOREIGN KEY (SID)
      REFERENCES Students(SID)
      ON DELETE CASCADE
      ON UPDATE CASCADE
);
```

In MySQL: RESTRICT | CASCADE | SET NULL | NO ACTION | SET DEFAULT

23

SQL – general rules

- SQL **commands** are case insensitive:
  - Same: SELECT, Select, select
  - Same: Students, studentS
- **Values** are **not:**
  - Different: 'Szczecin', 'szczecin'
- Use single quotes for constants:
  - 'abc' - yes
  - "abc" – no (some DBMS allows)

24

SQL – basic retrieval queries

List of attributes

```
SELECT <list of attributes>
FROM   <relation>
WHERE  <logical conditions>
;
```

Condition on records

Selection is the operation of filtering a relation's tuples on some condition

```
SELECT *
FROM   Students
WHERE  first_name = 'Mia';
```

Students

SID	first_name	last_name
12	Mia	Wallace
34	Vincent	Vega
55	Butch	Coolidge

Answer

SID	first_name	last_name
12	Mia	Wallace

25

SQL – basic retrieval queries

Projection is the operation of producing an output table with tuples that have a subset of their prior attributes.

Input schema: `Students(SID, fist_name, last_name)`

```
SELECT fist_name, last_name
FROM   Students
WHERE  first_name = 'Mia';
```

Output schema: `Answer(fist_name, last_name)`

Students

SID	first_name	last_name
12	Mia	Wallace
34	Vincent	Vega
55	Butch	Coolidge

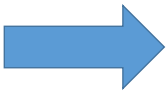
Answer

first_name	last_name
Mia	Wallace

26

SQL – basic retrieval queries

```
SELECT CID
FROM   Enrolled;
```



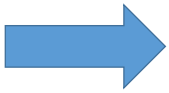
Answer1

CID
CS101
CS900
CS900
CS102

Enrolled

SID	CID	grade
12	CS101	NULL
12	CS900	5.0
55	CS900	NULL
34	CS102	3.5

```
SELECT DISTINCT CID
FROM   Enrolled;
```



Answer2

CID
CS101
CS900
CS102

27

SQL – pattern matching

```
SELECT fist_name, last_name
FROM   Students
WHERE  first_name = 'Mia';
```

```
SELECT fist_name, last_name
FROM   Students
WHERE  first_name LIKE '_a%';
```

% = any sequence of characters
_ = any single character
\ = an escape character: _, \%, \\

```
SELECT fist_name, last_name
FROM   Students
WHERE  first_name NOT LIKE '_a%';
```

28

SQL – arithmetic and logical operators

```
SELECT fist_name, last_name
FROM   Students
WHERE  first_name = 'Mia' OR last_name != 'Vega';
```

Logical statement

- =, !=, <>, >, <, >=, <=
- +, -, *, /
- AND (&&), OR (||)
- BETWEEN

```
SELECT *
FROM   Courses
WHERE  ECTS BETWEEN 3 AND 6;
```

Logical equivalent of
min <= expr AND expr <= max

29

SQL - ordering

```
SELECT   *
FROM     Enrolled
WHERE    SID = 34
ORDER BY grade, CID DESC;
```

Ties are broken by the second attribute on the ORDER BY list, etc.

Ordering is ascending, unless you specify the DESC keyword.

```
SELECT   grade, CID
FROM     Enrolled
WHERE    SID = 34
ORDER BY 1, 2 DESC;
```

30

SQL - JOINS

```
SELECT *
FROM   Students
JOIN   Enrolled ON Students.SID = Enrolled.SID;
```

=

```
SELECT *
FROM   Students, Enrolled
WHERE Students.SID = Enrolled.SID;
```

Students

SID	first_name	last_name
12	Mia	Wallace
34	Vincent	Vega
55	Butch	Coolidge

Enrolled

SID	CID	grade
12	CS101	NULL
12	CS900	5.0
55	CS900	NULL
34	CS102	3.5

A join between tables returns all unique combinations of their tuples **which meet some specified join condition**

31

SQL - JOINS

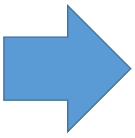
```
SELECT *
FROM   Students
JOIN   Enrolled ON Students.SID = Enrolled.SID;
```

Students

SID	first_name	last_name
12	Mia	Wallace
34	Vincent	Vega
55	Butch	Coolidge

Enrolled

SID	CID	grade
12	CS101	NULL
12	CS900	5.0
55	CS900	NULL
34	CS102	3.5



Answer

SID	first_name	last_name	SID	CID	grade
12	Mia	Wallace	12	CS101	NULL
12	Mia	Wallace	12	CS900	5.0
34	Vincent	Vega	34	CS102	3.5
55	Butch	Coolidge	55	CS900	NULL

32

SQL - aliases

```
SELECT Students.first_name, Enrolled.grade
FROM   Students
JOIN   Enrolled ON Students.SID = Enrolled.SID;
```

```
SELECT s.first_name, e.grade
FROM   Students s
JOIN   Enrolled e ON s.SID = e.SID;
```

```
SELECT s.first_name AS 'First name', e.grade AS StudentGrade
FROM   Students AS s
JOIN   Enrolled AS e ON s.SID = e.SID;
```

33

SQL - JOINS

```
SELECT s.last_name, e.grade
FROM   Students s
JOIN   Enrolled e ON s.SID = e.SID;
```

Students

SID	first_name	last_name
12	Mia	Wallace
34	Vincent	Vega
55	Butch	Coolidge

Enrolled

SID	CID	grade
12	CS101	NULL
12	CS900	5.0
55	CS900	NULL
34	CS102	3.5

1

Students × Enrolled

SID	first_name	last_name	SID	CID	grade
12	Mia	Wallace	12	CS101	NULL
12	Mia	Wallace	12	CS900	5.0
12	Mia	Wallace	55	CS102	NULL
12	Mia	Wallace	34	EN101	3.5
34	Vincent	Vega	12	CS101	NULL
34	Vincent	Vega	12	CS900	5.0
34	Vincent	Vega	55	CS102	NULL
34	Vincent	Vega	34	EN101	3.5
55	Butch	Coolidge	12	CS101	NULL
55	Butch	Coolidge	12	CS900	5.0
55	Butch	Coolidge	55	CS102	NULL
55	Butch	Coolidge	34	EN101	3.5

2

Join

SID	first_name	last_name	SID	CID	grade
12	Mia	Wallace	12	CS101	NULL
12	Mia	Wallace	12	CS900	5.0
34	Vincent	Vega	34	CS102	3.5
55	Butch	Coolidge	55	CS900	NULL

3

Projection

last_name	grade
Wallace	NULL
Wallace	5.0
Vega	3.5
Coolidge	NULL

34

SQL - JOINS

Find all names of students enrolled in courses?

Students

SID	first_name	last_name
12	Mia	Wallace
34	Vincent	Vega
55	Butch	Coolidge

Enrolled

SID	CID	grade
12	CS101	NULL
12	CS900	5.0
55	CS900	NULL
34	CS102	3.5

```
SELECT s.last_name
FROM   Students s
JOIN   Enrolled e ON s.SID = e.SID;
```

Answer1

last_name
Wallace
Wallace
Vega
Coolidge

What do we really expect to see in the results?

```
SELECT DISTINCT s.last_name
FROM   Students s
JOIN   Enrolled e ON s.SID = e.SID;
```

Answer2

last_name
Wallace
Vega
Coolidge

35

SQL – Nested loop join algorithm

```
SELECT *
FROM   Students
JOIN   Enrolled ON Students.SID = Enrolled.SID;
```

```
Answer = {}
for s in Students do
   for e in Enrolled do
      if s.SID = e.SID
         then Answer = Answer ∪ {(s.*, e.*)}
return Answer
```

multiset union

36

Multiset algebra

multiset | elements | distinct elements | multiplicity

- $A = \{a, b, a, c, b, a, c, a, c\} = \{(a, 4), (b, 2), (c, 3)\}$
- $A = \{ (a, m(a)) : a \in A,\ m: A \to \mathbb{Z}^+ \}$

- **Union**: $C = A \cup B \Rightarrow \{(x, m_A(x) + m_B(x)) : x \in a(A) \cup b(B) \}$
- $A \cup B = \{(a, 4), (b, 2), (c, 3)\} \cup \{(a, 2), (c, 5)\} = \{(a, 6), (b, 2), (c, 8)\}$
- $a: 4 + 2 = 6; b: 2 + 0 = 2; c: 3 + 5 = 8$

37

Multiset algebra

- **Intersection**: $C = A \cap B \Rightarrow \{ (x, \min(m_A(x), m_B(x))) : x \in a(A) \cup b(B)\}$
- $A \cap B = \{(a, 4), (b, 2), (c, 3)\} \cap \{(a, 2), (c, 5)\} = \{(a, 2), (c, 3)\}$
- $a: \min(4, 2) = 2; b: \min(2, 0) = 0; c: \min(3, 5) = 3$

- **Difference**: $C = A - B \Rightarrow \{ (x, \max(m_A(x) - m_B(x), 0)) : x \in a(A) \cup b(B)\}$
- $A - B = \{(a, 4), (b, 2), (c, 3)\} - \{(a, 2), (c, 5)\} = \{(a, 2), (b, 2)\}$
- $a: \max(4 - 2, 0) = 2; b: \max(2 - 0, 0) = 2; c: \max(3 - 5, 0) = 0$

38

Multiset Operations in SQL

A

Country
USA
Japan
Japan
Poland
Poland

B

Country
USA
Japan
Japan

```
SELECT A.Country AS Country
FROM A
UNION ALL
SELECT B.Country AS Country
FROM B;
```

Answer1

Country
USA
Japan
Japan
Poland
Poland
USA
Japan
Japan

```
SELECT A.Country AS Country
FROM A
UNION
SELECT B.Country AS Country
FROM B;
```

Answer2

Country
USA
Japan
Poland

Duplicates removed

39

Multiset Operations in SQL

A

Country
USA
Japan
Japan
Poland
Poland

B

Country
USA
Japan
Japan

```
SELECT A.Country AS Country
FROM A
INTERSECT
SELECT B.Country AS Country
FROM B;
```

Answer1

Country
USA
Japan

Duplicates removed

```
SELECT A.Country AS Country
FROM A
EXCEPT
SELECT B.Country AS Country
FROM B;
```

Answer2

Country
Poland

Duplicates removed

40

Null

Null value can have various interpretations:

- value unknown,
- value exists but is not available,
- attribute does not apply to this tuple (aka value undefined).

Due to Null values in databases, we have three-valued logic (True, False, Null) instead of Boolean logic (True, False).

41

Null and Three-Valued Logic

- *For numerical operations,* NULL → NULL:
  - If $x = \text{NULL}$ then $(4 \cdot (3 - x) - 5)$ is still NULL
- If $x = \text{NULL}$ then $x = \text{'Joe'}$ is UNKNOWN

- *For boolean operations,* in SQL there are three values:
  FALSE = 0
  NULL = 0.5
  TRUE = 1

42

42

NULL and Three-Valued Logic

- p AND q = min(p, q)
- p OR q = max(p, q)
- NOT p = 1 – p

p	q	p OR q	p AND q	p = q
True	True	True	True	True
True	False	True	False	False
True	NULL	True	NULL	NULL
False	True	True	False	False
False	False	False	False	True
False	NULL	NULL	False	NULL
NULL	True	True	NULL	NULL
NULL	False	NULL	False	NULL
NULL	NULL	NULL	NULL	NULL

43

43

NULL and Three-Valued Logic

Enrolled

SID	CID	grade
12	CS101	NULL
12	CS900	5.0
55	CS900	NULL
34	CS102	3.5

```
SELECT   *
FROM     Enrolled
WHERE    grade IS NULL;
```

SID	CID	grade
12	CS101	NULL
55	CS900	NULL

```
SELECT   *
FROM     Enrolled
WHERE    grade IS NOT NULL;
```

SID	CID	grade
12	CS900	5.0
34	CS102	3.5

44

Aggregation

Enrolled

SID	CID	grade
12	CS101	NULL
12	CS900	5.0
55	CS900	NULL
34	CS102	3.5

```
SELECT COUNT(grade),
       COUNT(*),
       COUNT(DISTINCT grade)
       SUM(grade),
       MAX(7 - grade),
       MIN(grade * (2 + grade)),
       AVG(grade),
       STD(grade),
       VARIANCE(grade)
FROM   Enrolled
WHERE  SID = 12;
```

Number of rows without NULLs for the attribute 'grade'

Number of rows in the table

Number of unique values for the attribute

Any arithmetical expression

45

Grouping

Enrolled

SID	CID	grade
12	CS101	NULL
12	CS900	5.0
55	CS900	NULL
34	CS102	3.5

```
SELECT    SID, COUNT(*), AVG(grade)
FROM      Enrolled
GROUP BY  SID;
```

SID	CID	grade
12	CS101	NULL
12	CS900	5.0

→ (2, 5)

SID	CID	grade
55	CS900	NULL

→ (1, NULL)

SID	CID	grade
34	CS102	3.5

→ (1, 3.5)

→

SID	COUNT(*)	AGV(grade)
12	2	5
55	1	NULL
34	1	3.5

46

Having

```
SELECT    SID, COUNT(*), AGV(grade)
FROM      Enrolled
WHERE     SID > 10
GROUP BY  SID
HAVING    COUNT(*) > 1;
```

Any condition with an aggregation function

Enrolled

SID	CID	grade
12	CS101	NULL
12	CS900	5.0
55	CS900	NULL
34	CS102	3.5

SID	CID	grade
12	CS101	NULL
12	CS900	5.0

→ (2, 5)

SID	CID	grade
55	CS900	NULL

→ (1, N L) ✗

SID	CID	grade
34	CS102	3.5

→ (1, 3) ✗

→

SID	COUNT(*)	AGV(grade)
12	2	5

47

Sequence of SELECT query steps

1. Evaluate FROM, JOIN, WHERE: apply condition on the attributes, record level
2. GROUP BY the attributes
3. Apply condition to each group HAVING
4. Compute and project SELECT
5. ORDER BY and return the result

```
SELECT    a, b, c
FROM      Relation1 r1
JOIN      Relation2 r2
    ON r1.a = f2.a
WHERE     b + c > 0
GROUP BY  a
HAVING    COUNT(*) > 1
ORDER BY  1 DESC, 2;
```

HAVING clauses contains conditions on **aggregates**

WHERE clauses condition on **individual tuples…**

48

Multi table joins

```
SELECT Students.first_name, Students.last_name,
       Enrolled.grade,
       Courses.cname, Courses.ECTS
FROM   Students
JOIN   Enrolled ON Students.SID = Enrolled.SID
JOIN   Courses ON Enrolled.CID = Courses.CID;
```

Students

SID	first_name	last_name
12	Mia	Wallace
34	Vincent	Vega
55	Butch	Coolidge

Enrolled

SID	CID	grade
12	CS101	NULL
12	CS900	5.0
55	CS900	NULL
34	CS102	3.5

Courses

CID	cname	ECTS
CS101	Programming	5
CS900	Databases	6
CS102	Algorithms	4
EN101	Mechanics	5

49

Left join

```
SELECT Students.first_name, Students.last_name,
       Enrolled.CID, Enrolled.grade
FROM   Students
LEFT JOIN Enrolled ON Students.SID = Enrolled.SID;
```

Students

SID	first_name	last_name
12	Mia	Wallace
34	Vincent	Vega
55	Butch	Coolidge
77	Jules	Winnfield

Enrolled

SID	CID	grade
12	CS101	NULL
12	CS900	5.0
55	CS900	NULL
34	CS102	3.5

Answer

SID	first_name	last_name	CID	grade
12	Mia	Wallace	CS101	NULL
12	Mia	Wallace	CS900	5.0
34	Vincent	Vega	CS900	NULL
55	Butch	Coolidge	CS102	3.5
77	Jules	Winnfield	NULL	NULL

50

Left join

```
SELECT Students.first_name, Students.last_name,
       Enrolled.CID, Enrolled.grade
FROM Enrolled
LEFT JOIN Students ON Students.SID = Enrolled.SID;
```

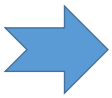
Tables swapped places, now Enrolled is to the left.

Students

SID	first_name	last_name
12	Mia	Wallace
34	Vincent	Vega
55	Butch	Coolidge
77	Jules	Winnfield

Enrolled

SID	CID	grade
12	CS101	NULL
12	CS900	5.0
55	CS900	NULL
34	CS102	3.5



Answer

SID	first_name	last_name	CID	grade
12	Mia	Wallace	CS101	NULL
12	Mia	Wallace	CS900	5.0
34	Vincent	Vega	CS900	NULL
55	Butch	Coolidge	CS102	3.5

51

Right join

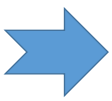
```
SELECT Enrolled.CID, Enrolled.grade,
       Courses.cname, Courses.ECTS
FROM   Enrolled
RIGHT JOIN Courses ON Courses.CID = Enrolled.CID;
```

Enrolled

SID	CID	grade
12	CS101	NULL
12	CS900	5.0
55	CS900	NULL
34	CS102	3.5

Courses

CID	cname	ECTS
CS101	Programming	5
CS900	Databases	6
CS102	Algorithms	4
EN101	Mechanics	5
EL300	Electronics	6



Answer

CID	grade	cname	ECTS
CS101	NULL	Programming	5
CS900	5.0	Databases	6
CS900	NULL	Databases	6
CS102	3.5	Algorithms	4
NULL	NULL	Mechanics	5
NULL	NULL	Electronics	6

52

Right join

```
SELECT Enrolled.CID, Enrolled.grade,
       Courses.cname, Courses.ECTS
FROM   Enrolled
RIGHT JOIN Courses ON Courses.CID = Enrolled.CID
WHERE  Enrolled.CID IS NULL;
```

Only courses without enrolled students

Enrolled

SID	CID	grade
12	CS101	NULL
12	CS900	5.0
55	CS900	NULL
34	CS102	3.5

Courses

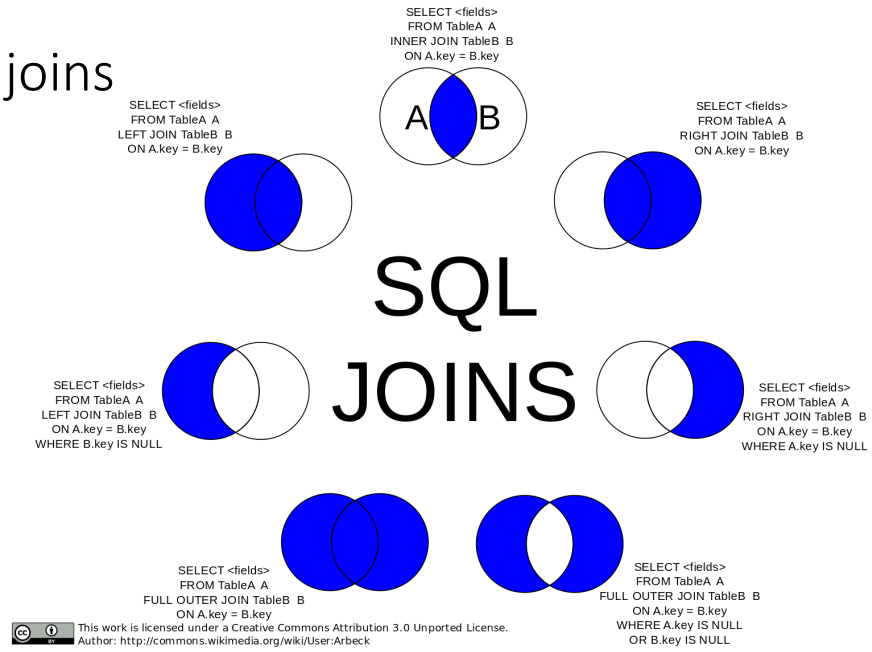
CID	cname	ECTS
CS101	Programming	5
CS900	Databases	6
CS102	Algorithms	4
EN101	Mechanics	5
EL300	Electronics	6

Answer

CID	grade	cname	ECTS
NULL	NULL	Mechanics	5
NULL	NULL	Electronics	6

53

Types of joins



54

Subqueries (nested queries)

```
SELECT Enrolled.CID, Enrolled.grade,
       (SELECT COUNT(DISTINCT *) FROM Students) AS SubQuery1
FROM   Enrolled
JOIN (SELECT cname, CID FROM Courses WHERE CID LIKE 'CS%') Subquery2
  ON Enrolled.CID = Subquery2.CID
WHERE  grade > (SELECT AVG(grade) FROM Enrolled);
```

Only one value as a result of a SELECT subquery

A table with a joining attribute in JOIN subquery

Only one value as a result of a WHERE subquery

Enrolled

SID	CID	grade
12	CS101	NULL
12	CS900	5.0
55	CS900	NULL
34	CS102	3.5

- We can do nested queries because SQL is ***compositional***
- Everything (inputs / outputs) is represented as multisets
- The output of one query can thus be used as the input to another (nesting)!
- This is extremely powerful!

55

Subqueries (nested queries)

Often there is more than one way to write a query in SQL

```
SELECT e.CID, e.grade
FROM   Enrolled e
WHERE  CID IN (
  SELECT CID
  FROM Courses
  WHERE ECTS >= 5);
```

=

```
SELECT e.CID, e.grade
FROM   Enrolled e
RIGHT JOIN Courses c ON c.CID = e.CID
WHERE c.ECTS >= 5;
```

=

```
SELECT e.CID, e.grade
FROM   Enrolled e
WHERE  CID NOT IN (
  SELECT CID
  FROM Courses
  WHERE ECTS < 5);
```

=

```
SELECT e.CID, e.grade
FROM   Enrolled e
WHERE  EXISTS (
  SELECT *
  FROM Courses c
  WHERE ECTS >= 5 AND c.CID = e.CID);
```

Also, there is NOT EXISTS

56

Alternatives to INTERSECT and EXCEPT

INTERSECT and EXCEPT are not supported by some DBMS

```
SELECT A.Country AS Country
FROM A
INTERSECT
SELECT B.Country AS Country
FROM B;
```

=

```
SELECT A.Country AS Country
FROM   A
WHERE EXISTS(
      SELECT *
      FROM B
       WHERE A. Country = B. Country)
```

```
SELECT A.Country AS Country
FROM A
EXCEPT
SELECT B.Country AS Country
FROM B;
```

=

```
SELECT A.Country AS Country
FROM   A
WHERE NOT EXISTS(
      SELECT *
       FROM B
       WHERE A. Country = B. Country)
```

57

Correlated nested queries

```
Movie(title, year, director, length)
```

```
SELECT DISTINCT title
FROM   Movie AS m
WHERE  year <> ANY(
          SELECT  year
          FROM    Movie AS x
          WHERE   x.title =  m.title)
```

Find movies whose title appears more than once.

Note the scoping of the variables!

58

Views (virtual tables)

```
CREATE VIEW view_name AS
SELECT E.CID, E.grade, C.cname, C.ECTS
FROM Enrolled E
RIGHT JOIN Courses C ON C.CID = E.CID;
```

- A view is a single table that is derived from other tables.
- These other tables can be base tables or previously defined views.
- A view does not necessarily exist in physical form; it is a virtual table, in contrast to base tables, whose tuples are always physically stored in the database.

```
SELECT CID, cname
FROM view_name
WHERE AGV(grade) < 3;
```

59

Views – data updates

```
UPDATE View_name
SET cname = 'Data management'
WHERE cname = 'Databases';
```

What happens?
Will the data in the Courses table be changed?

- A view with a single defining table is updatable if the view attributes contain the primary key of the base relation, as well as all attributes with the NOT NULL constraint that do not have default values specified.
- Views defined on multiple tables using joins are generally not updatable.
- Views defined using grouping and aggregate functions are not updatable.

60

Materialized views

```
CREATE MATERIALIZED VIEW View_name AS
SELECT Enrolled.CID, Enrolled.grade,
       Courses.cname, Courses.ECTS
FROM   Enrolled
RIGHT JOIN Courses ON Courses.CID = Enrolled.CID;
```

- Physical copy of data
- Update strategies:
  - **immediate update** strategy updates a view as soon as the base tables are changed
  - **lazy update** strategy updates the view when needed by a view query
  - **periodic update** strategy updates the view periodically
- Implemented in some DBMS: Oracle, PostgreSQL, SQL Server

61

CTE - Common Table Expressions

A common table expression (CTE) is a named temporary result set that exists within the scope of a single statement and that can be referred to later within that statement.

The number of attributes in the list must be the same as the number of columns in the result set.

```
WITH cte_name (CourseID, Grade) AS
      (SELECT Enrolled.CID, Enrolled.grade,
      FROM   Enrolled
      RIGHT JOIN Courses ON Courses.CID = Enrolled.CID);
```

```
SELECT CourseID, Grade FROM cte_name;
```

62

Recursive CTE

```
WITH RECURSIVE fibonacci(n, fib_n, next_fib_n) AS
(
SELECT 1 , 0, 1
UNION  ALL
SELECT n+1, next_fib_n, fib_n + next_fib_n
FROM fibonacci
WHERE n < 10
)

SELECT * FROM fibonacci

SET SESSION cte_max_recursion_depth = 100;

SELECT /*+ MAX_EXECUTION_TIME(1000) */ * FROM fibonacci;
```

Anchor part

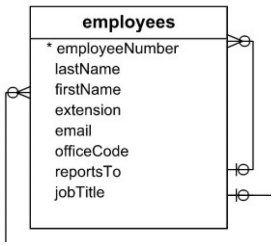
Recursive part

Usage

n	fib_n	next_fib_n
1	0	1
2	1	1
3	1	2
4	2	3
5	3	5
6	5	8
7	8	13
8	13	21
9	21	34
10	34	55
11	55	89
12	89	144
13	144	233
14	233	377
15	377	610

63

Recursive CTE




```
WITH RECURSIVE Hierarchy AS
(
SELECT employeeNumber, lastName,
       CAST(lastName AS CHAR(100)) AS path
FROM employees
WHERE reportsTo IS  NULL
UNION  ALL
SELECT e.employeeNumber, e.lastName,
       CONCAT(h.path, ',', e.lastName) AS path
FROM Hierarchy h
JOIN employees e ON e.reportsTo = h.employeeNumber
)

SELECT * FROM Hierarchy
```

employeeNumber	lastName	path
1002	Murphy	Murphy
1056	Patterson	Murphy, Patterson
1076	Firrelli	Murphy, Firrelli
1088	Patterson	Murphy, Patterson, Patterson
1102	Bondur	Murphy, Patterson, Bondur
1143	Bow	Murphy, Patterson, Bow
1621	Nishi	Murphy, Patterson, Nishi
1611	Fixter	Murphy, Patterson, Patterson, Fixter
1612	Marsh	Murphy, Patterson, Patterson, Marsh
1619	King	Murphy, Patterson, Patterson, King
1337	Bondur	Murphy, Patterson, Bondur, Bondur
1370	Hernandez	Murphy, Patterson, Bondur, Hernandez
1401	Castillo	Murphy, Patterson, Bondur, Castillo
1501	Bott	Murphy, Patterson, Bondur, Bott
1504	Jones	Murphy, Patterson, Bondur, Jones
1702	Gerard	Murphy, Patterson, Bondur, Gerard
1165	Jennings	Murphy, Patterson, Bow, Jennings
1166	Thompson	Murphy, Patterson, Bow, Thompson
1188	Firrelli	Murphy, Patterson, Bow, Firrelli
1216	Patterson	Murphy, Patterson, Bow, Patterson
1286	Tseng	Murphy, Patterson, Bow, Tseng
1323	Vanauf	Murphy, Patterson, Bow, Vanauf
1625	Kato	Murphy, Patterson, Nishi, Kato

64

Thank you for your attention!

Time for questions

65

65